BATTLESHIP MULTIPLAYER

KẾT NỐI SUPABASE VÀ WEB SERVICE

Môn Lập trình mạng căn bản (NT106) · UIT

Hoàng (thuyết trình chính) · Đức · Dũng

Mục Lục Nội Dung

Tổng Quan Đồ Án

Cloud và Web Service

Kịch Bản Demo

Nhóm Thực Hiện

Đồ Án Battleship Là Gì?

Trò chơi bắn tàu nhiều người chơi, theo lượt

- Đăng ký, đăng nhập, danh sách **Online/Offline**, chat sảnh.
- **Tạo phòng** bằng ID, ghép trận tự động (**Matchmaking**).
- Xếp tàu trên lưới 10×10, bắn theo lượt, **Server là bên quyết định** Hit/Miss.
- Lưu lịch sử đấu, thắng/thua và điểm xếp hạng.

Công nghệ cốt lõi của môn NT106

C# .NET 8 · `TcpListener/TcpClient` · async/await · framing 4-byte · JSON · AES-128 · BCrypt · EF Core

Tình Trạng Trước Khi Thêm Cloud

Đã có

- TCP Server bất đồng bộ + **PacketRouter** đăng ký handler.
- Giao thức: framing 4-byte + **Handshake** kiểm tra phiên bản.
- **Mã hóa AES-128** toàn bộ gói tin.
- Đăng nhập/đăng ký với mật khẩu băm **BCrypt**.
- Dữ liệu lưu **SQLite** cục bộ qua EF Core.

Hạn chế cần giải quyết

- Dữ liệu nằm trên **một máy**, chưa dùng chung.
- Chưa có **web service** để truy vấn dữ liệu từ bên ngoài.

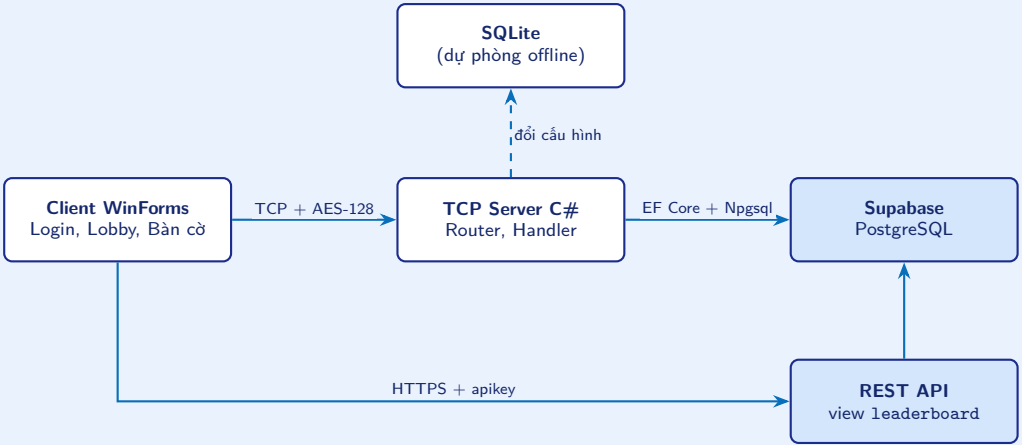
Vì Sao Chọn Supabase?

Tiêu chí	SQLite	Firebase	Supabase
Loại dữ liệu	File cục bộ (SQL)	NoSQL (Firestore)	**PostgreSQL** (SQL)
Dùng được EF Core hiện có	Có	**Không** (không có provider)	**Có** (Npgsql)
Web service có sẵn	Không	Có (SDK/REST)	**Có, REST tự sinh**
Cần Internet	Không	Có	Có
Công sức chuyển đổi	—	Rất lớn (viết lại lớp dữ liệu)	**Nhỏ** (đổi provider)

Kết luận

Supabase giữ nguyên mô hình quan hệ (Users, Matches, MatchDetails) và EF Core đã có, đồng thời cho **REST API** miễn phí. **SQLite vẫn giữ làm dự phòng** khi mất mạng.

Kiến Trúc Sau Khi Thêm Cloud



Đường game đi qua **TCP Server**; đường web service đi thẳng từ Client tới **REST của Supabase**.

Kết Nối Cơ Sở Dữ Liệu Với EF Core

Đổi nơi lưu trữ chỉ bằng cấu hình, không sửa code khi demo

- `BATTLESHIP_DB=supabase` → Server dùng **PostgreSQL** (Npgsql).
- `BATTLESHIP_DB=sqlite` (mặc định) → dùng `battleship.db` cục bộ.
- Chuỗi kết nối đọc từ biến môi trường, **không nằm trong mã nguồn**.
- Server **tự tạo bảng** (`EnsureCreated`) khi khởi động.

Lưu ý kỹ thuật

Dùng chuỗi kết nối **Session pooler** của Supabase (hỗ trợ IPv4) và `SSL Mode=Require`.

Web Service: REST API Của Supabase

Mỗi bảng/view trong Supabase tự có một endpoint

```
GET https://<project>.supabase.co/rest/v1/leaderboard
    ?select=*&order=Score.desc&limit=10
```

Header bắt buộc

- `apikey: <publishable key>` (bắt buộc)
- Thiếu khóa → **401**; khóa đúng nhưng chưa có bảng → 404

Phản hồi (JSON)

```
[{"Username":"admiral_a",
  "Wins":12,"Losses":3,
  "Score":1200}, ...]
```

Client C# gọi bằng `HttpClient` và hiển thị lên `ListView` **Bảng xếp hạng**.

Cơ Chế Đăng Nhập An Toàn

Nguy cơ	Cách phòng thủ
Gửi gói tin khi chưa đăng nhập	Router **chặn** mọi gói, trừ Handshake/Login/Register
Dò mật khẩu hàng loạt	**Khóa 60 giây** sau 5 lần sai (theo IP và theo tài khoản)
Dữ liệu đầu vào xấu	Kiểm tra tên 3–20 ký tự, mật khẩu 6–64 ký tự
Một tài khoản đăng nhập nhiều nơi	Mỗi tài khoản **một phiên** tại một thời điểm
Giả mạo *Admin* và *admin*	So sánh tên **không phân biệt hoa thường**
Đoán tên tài khoản qua thời gian trả lời	Luôn chạy BCrypt, thông báo lỗi **chung chung**
Lộ mật khẩu khi lưu trữ	Chỉ lưu chuỗi băm **BCrypt** (có muối)

Kiểm thử

19 kịch bản tấn công qua TCP, chạy trên server thật: **đều đạt**.

Bảo Mật Dữ Liệu

Nhiều lớp bảo vệ

- **Trên đường truyền game:** gói tin TCP mã hóa **AES-128**.
- **Mật khẩu:** chỉ lưu chuỗi băm **BCrypt**, không lưu chữ rõ.
- **Supabase:** bật **Row Level Security**, thu quyền khóa công khai trên bảng gốc.
- **Web service:** chỉ mở **view** `leaderboard` (không có cột mật khẩu).
- **Khóa:** chỉ `publishable key` ở Client; `service_role key` và mật khẩu DB nằm trong biến môi trường, **không commit**.

Vì sao phải khóa bảng gốc?

Supabase mặc định mở REST cho mọi bảng. Nếu không bật RLS, ai có `publishable key` đều đọc được cột mật khẩu băm của mọi người chơi.

Demo Chứng Minh "Móc Được Dữ Liệu"

1. **Ghi:** đăng ký tài khoản trong game → mở **Supabase Table Editor**, thấy dòng mới, cột mật khẩu là chuỗi `$2a$...` (BCrypt).
2. **Đọc qua web service:** gọi `GET .../rest/v1/leaderboard` → nhận **JSON**.
3. **Đọc trong ứng dụng:** Lobby → nút **Bảng xếp hạng** hiển thị dữ liệu lấy qua REST.
4. **Dự phòng:** đặt `BATTLESHIP_DB=sqlite` → game vẫn chạy khi mất mạng.

Thời lượng

Khoảng **5 phút**. Hoàng thuyết trình và điều khiển demo, Đức và Dũng thao tác máy thứ hai.

Rủi Ro Và Phương Án Dự Phòng

Rủi ro	Phương án
Mạng trường chập chờn	Hotspot 4G; chuyển sang **SQLite**; phát **video demo** quay sẵn
Supabase gói miễn phí tạm dừng project	Kiểm tra dashboard trước buổi demo, bấm **Restore** nếu cần
Chuỗi kết nối *direct* không nối được (IPv6)	Dùng chuỗi **Session pooler**
Lộ khóa hoặc mật khẩu	Biến môi trường, không commit, không dùng `service_role` ở Client

Phân Công Thuyết Trình Và Thực Hiện

Thành viên	Phần thuyết trình	Phần kỹ thuật
Hoàng CORE THUYẾT TRÌNH	Mở đầu, kiến trúc, bảo mật, **demo live**, kết luận	Supabase, chuyển DB (SQLite ↔ Supabase), TCP/AES, Bảng xếp hạng qua REST
Dũng	Mục *Cơ sở dữ liệu* (ERD, RLS), thao tác máy demo	Models/EF Core, SQL `setup`, dữ liệu mẫu, nút Bảng xếp hạng ở Lobby
Đức	Mục *Kiến trúc server và game*, thao tác máy thứ hai	RoomManager, State Machine, video demo dự phòng

Hoàng nói chính; Đức và Dũng trả lời câu hỏi thuộc mảng của mình.

Kết Luận

Những gì đạt được

- Game dùng **raw socket TCP** với giao thức tự thiết kế, mã hóa **AES-128**.
- **Đăng nhập an toàn:** BCrypt, chặn gói khi chưa đăng nhập, khóa khi dò mật khẩu.
- Dữ liệu lưu được trên **Supabase (PostgreSQL)** qua EF Core.
- Có **web service REST** để truy vấn bảng xếp hạng từ bên ngoài.
- Vẫn chạy được **offline** nhờ SQLite dự phòng.

Hướng phát triển

Đăng nhập bằng token của Supabase Auth, đồng bộ lịch sử trận đấu lên cloud, thêm bảng xếp hạng theo tuần.

CẢM ƠN ĐÃ THEO DÕI!

Nhóm sẵn sàng trả lời câu hỏi